CHRISTOPHER NEWPORT UNIVERSITY · SCHOOL OF ENGINEERING & COMPUTING

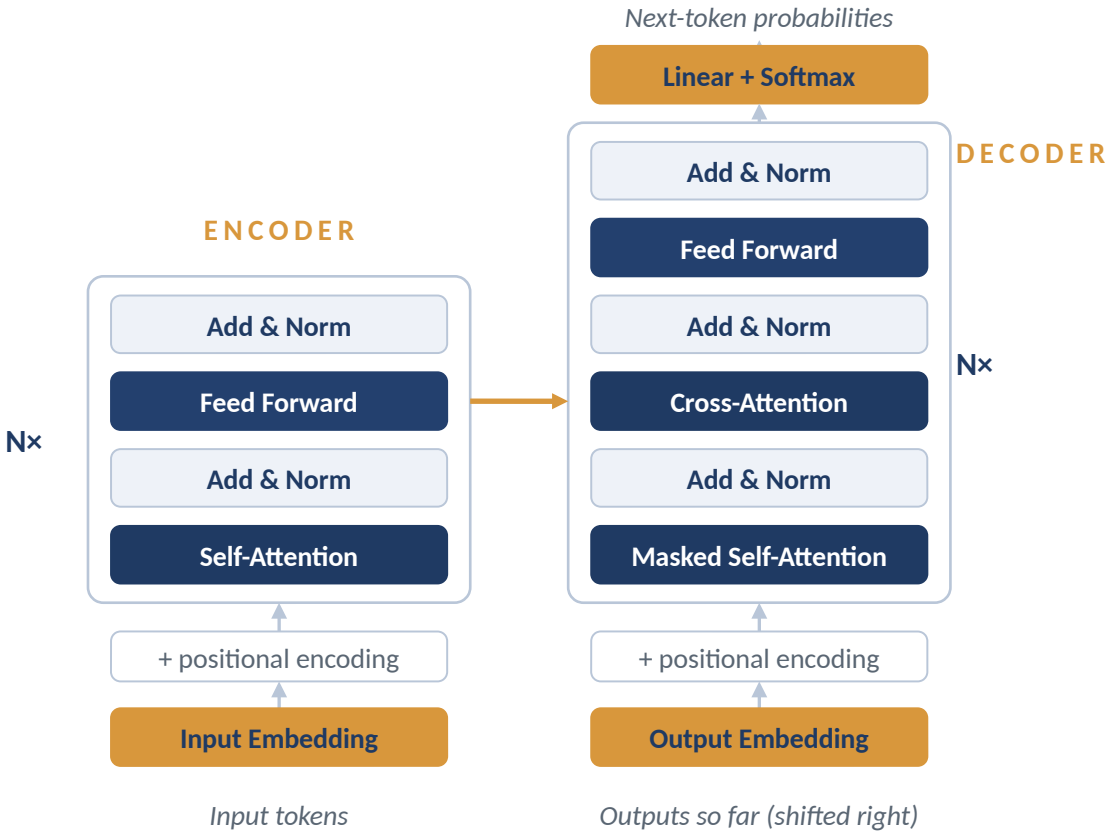
Inside the

Transformer

What the encoder and the decoder are each for, and how to build on an encoder

Keith Perkins · School of Engineering & Computing

The original transformer (2017)



BUILT FOR TRANSLATION

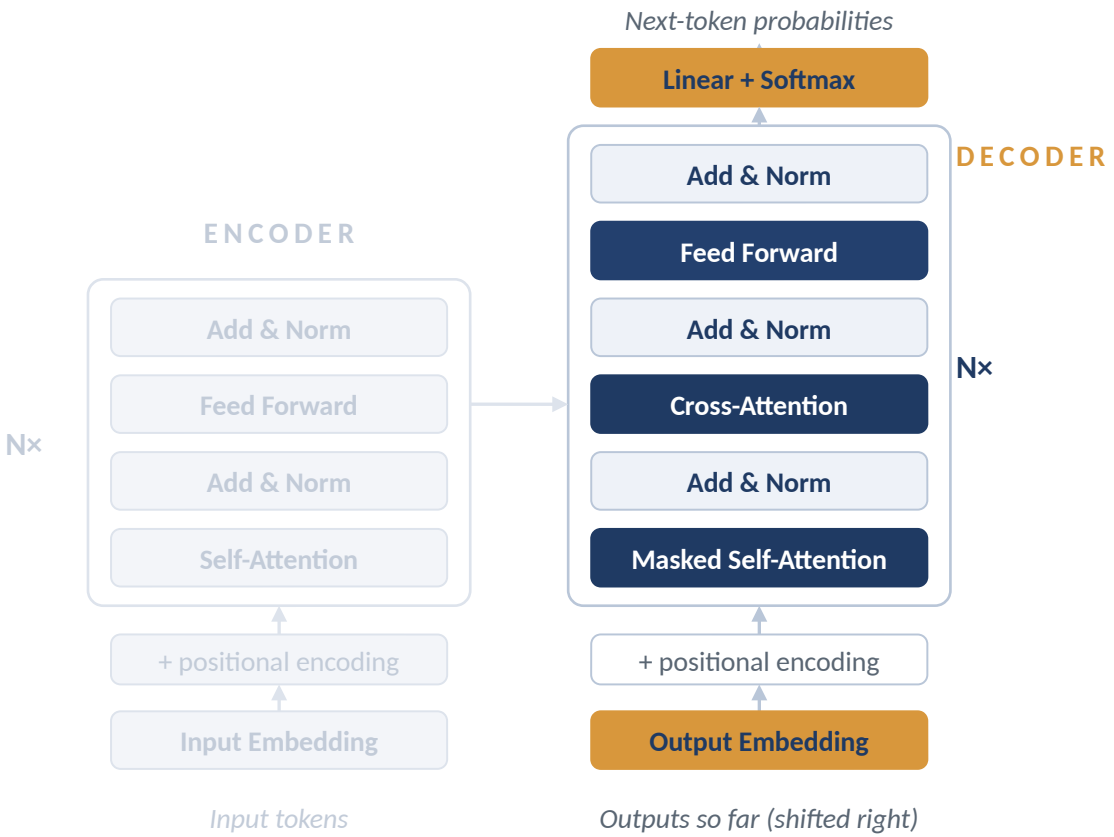
A sequence-to-sequence model: it reads a sentence in one language and writes it in another.

The bat flew → El murciélago voló

Encoder (left) Reads the whole input at once and turns each token into a context-aware vector

Decoder (right) Writes the output one token at a time, using what it has written so far and the encoder's vectors (gold arrow)

The decoder writes



Predicts the next token, appends it, and repeats: generation is token by token (a serial process).

USED FOR

Chatbots and assistants
ChatGPT, Claude, Gemini

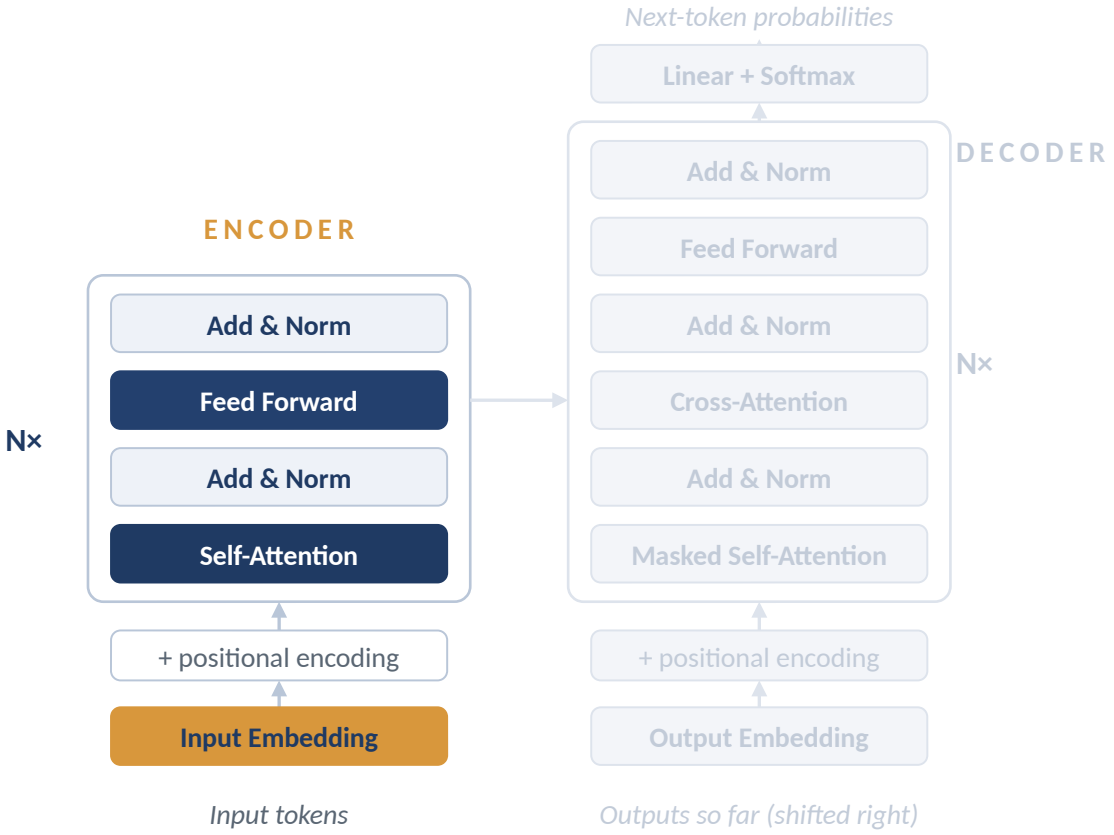
Writing and summarizing
drafts, emails, reports

Code generation
completion, whole functions

Translation by prompting
"Translate to Spanish: ..."

Masked self-attention: each token sees only the tokens before it. We will not do much with decoders in this course.

The encoder reads



Reads the whole input at once (a parallel process) and outputs one context-aware vector per token.

USED FOR

Classification
sentiment, spam, topic

Embeddings for search
semantic search, RAG

Named entities
people, places, organizations

Question answering
find the answer span in a text

Self-attention with no mask: every token sees every other token, before and after it. We will mostly use encoders for the rest of this course.

WHY CONTEXT MATTERS

Same word, different meanings

The bat flew out of the cave.

bat = an animal

He swung the bat at the ball.

bat = a piece of sports equipment

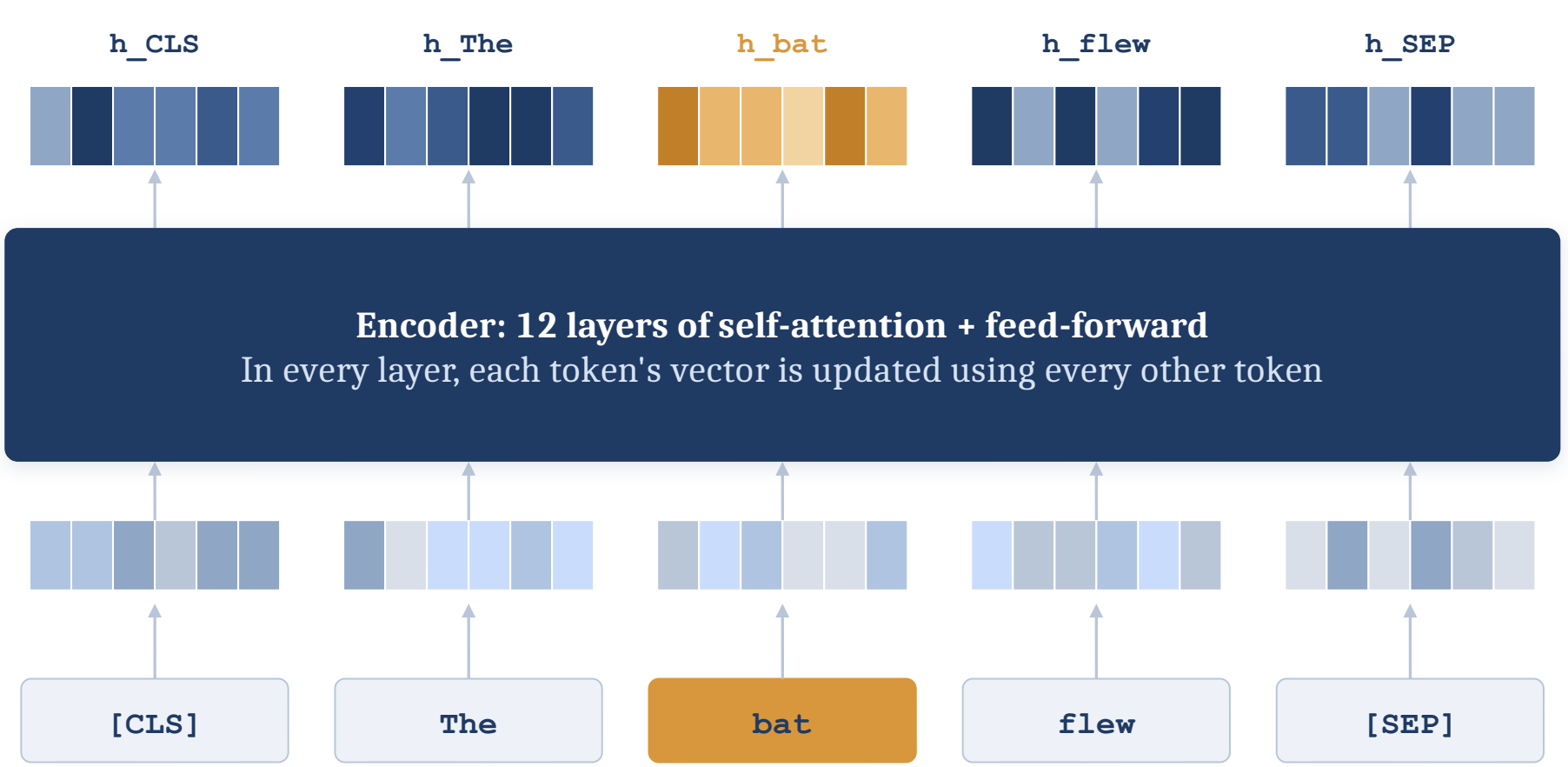
A PLAIN EMBEDDING TABLE: ONE FIXED VECTOR PER WORD



The embedding layer looks words up in a table, so it cannot tell the animal from the club. Self-attention fixes this: each token's vector is rebuilt from the words around it.

INSIDE THE ENCODER

"The bat flew" goes in, contextual embeddings come out



Now "bat" depends on its sentence

WHERE THE VECTORS LAND (ILLUSTRATIVE 2-D VIEW)



WHAT CHANGED

h_bat from "The bat flew" lands near animal words.

h_bat from "He swung the bat" lands near baseball words.

The static table vector sits in between, a blur of both meanings.

And h_CLS? A vector for the whole sentence. We will use it next.

TRY IT

Get the contextual embeddings yourself

PYTHON · HUGGING FACE TRANSFORMERS

```
from transformers import AutoTokenizer, AutoModel
import torch

tok  = AutoTokenizer.from_pretrained("bert-base-uncased")
bert = AutoModel.from_pretrained("bert-base-uncased")

inputs = tok("The bat flew", return_tensors="pt")
# tokens: [CLS] the bat flew [SEP]

with torch.no_grad():
    out = bert(**inputs)
h = out.last_hidden_state   # shape [1, 5, 768]
h_bat = h[0, 2]             # the vector for 'bat'
h_cls = h[0, 0]             # the [CLS] vector
```

ONE VECTOR PER TOKEN

last_hidden_state is the encoder's output: 5 tokens × 768 numbers.

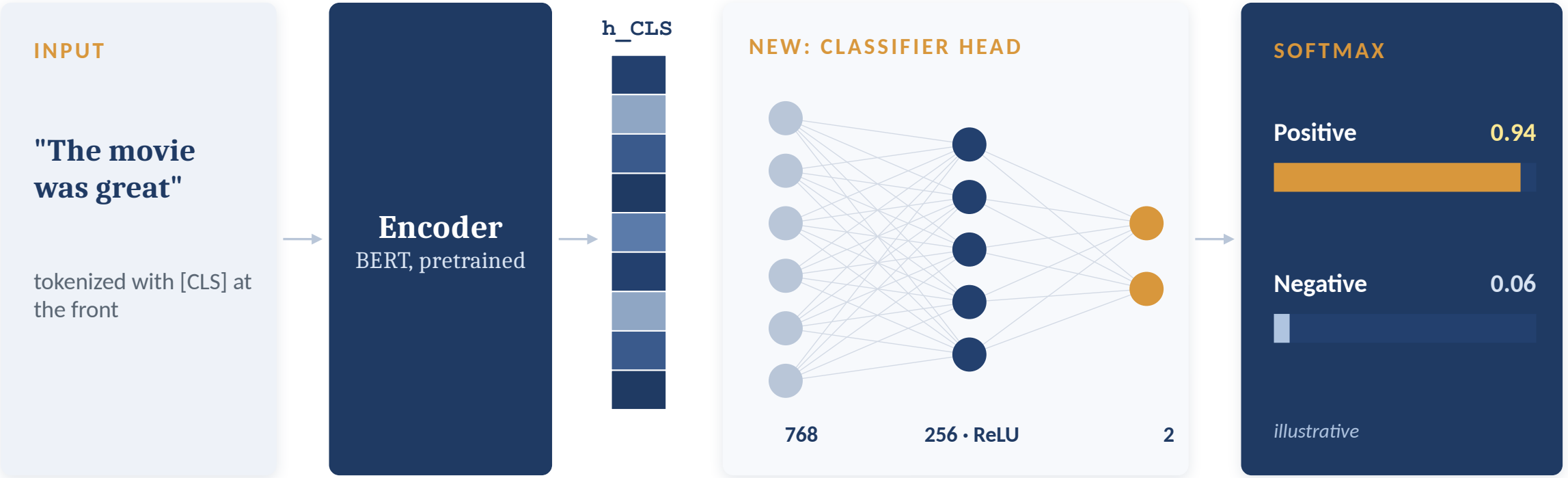
INDEX BY POSITION

[CLS] is position 0, so "bat" is position 2.

EXERCISE

Repeat for "He swung the bat" and compare the two h_bat vectors with cosine similarity.

Add a small neural network for sentiment



Only the head is new. Train it on labeled examples (movie reviews marked positive or negative) with cross-entropy loss. The encoder already understands the sentence; the head learns what to do with that understanding.

The classifier in PyTorch

PYTHON · PYTORCH + TRANSFORMERS

```python
class SentimentClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.bert = AutoModel.from_pretrained("bert-base-uncased")
        self.head = nn.Sequential(
            nn.Linear(768, 256), nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(256, 2))

    def forward(self, input_ids, attention_mask):
        h = self.bert(input_ids=input_ids,
                      attention_mask=attention_mask
                      ).last_hidden_state
        cls = h[:, 0]           # [batch, 768]
        return self.head(cls)   # logits [batch, 2]
```

TWO WAYS TO TRAIN IT

Freeze the encoder

Train only the head. Fast, works on a laptop, decent accuracy.

Fine-tune everything

Update BERT too, with a small learning rate (~2e-5). Slower, usually more accurate.

AND MORE

Same encoder, different heads

Keep the pretrained encoder. Change which output vectors you use and what small network sits on top.

USES H_CLS

Classification

Sentiment, spam, topic, toxicity → one label per text

USES H_CLS OF "A [SEP] B"

Sentence pairs

Is B a paraphrase of A? Does A imply B? Duplicate questions

USES EVERY TOKEN

Named entities

One label per token: PERSON, ORG, LOCATION, or none

USES EVERY TOKEN

Question answering

Two scores per token: where the answer starts and ends

AVERAGES EVERY TOKEN

Embeddings for search

One vector per sentence; compare with cosine similarity (SBERT, RAG)

USES MASKED TOKENS

Fill in the blank

Predict a hidden word: this is how BERT was pretrained

ENCODER MODELS YOU WILL MEET

BERT and its family

2018

BERT

The original. Base: 12 layers, 110M parameters

2019

RoBERTa

Same design, trained longer on more data

2019

DistilBERT

40% smaller and 60% faster, keeps ~97% of the accuracy

2019

SBERT

Tuned to produce sentence embeddings for search

2024

ModernBERT

Updated architecture, 8K-token context

We will mostly use encoders for the rest of this course. All of them load with the same AutoModel code you just saw.

SUMMARY

Key takeaways

The decoder writes

Generates text one token at a time. Chatbots, writing, and code.

The encoder reads

Sees the whole input at once and outputs one context-aware vector per token.

Context changes the vector

"bat" in "The bat flew" and "He swung the bat" get different embeddings.

Add a head for any task

A small network on top of the encoder's vectors: sentiment, entities, QA, search.

Next: fine-tune an encoder on a real dataset in the notebook.